

AI/BigData 인텐시브 과정

강사장철원

Section 0

코스소개

DAY1

DAY2

DAY3

DAY4

DAY5

데이터 분석을 위한 기초 수학

데이터 집계 및 시각화

DAY6

DAY7

DAY8

DAY9

DAY10

미니
프로젝트

확률분포, AB테스트

DAY11

DAY12

DAY13

DAY14

DAY15

머신러닝

DAY16

DAY17

DAY18

DAY19

DAY20

딥러닝

파이널 프로젝트

□ Pytorch 기초

AI/BigData 인텐시브 과정

Section 1. PyTorch 기초

Section 1-1. Numpy와 PyTorch의 차이

1D array - Numpy

```
In [1]: 1 import numpy as np
        2 import torch
```

```
In [2]: 1 a1 = np.array([1,2,3,4,5,6])
        2 print(a1)
        [1 2 3 4 5 6]
```

```
In [3]: 1 a1.ndim
        Out[3]: 1
```

```
In [4]: 1 a1.shape
        Out[4]: (6,)
```

```
In [5]: 1 a1[0]
        Out[5]: 1
```

```
In [6]: 1 a1[1]
        Out[6]: 2
```

```
In [7]: 1 a1[-1]
        Out[7]: 6
```

```
In [8]: 1 a1[2:5]
        Out[8]: array([3, 4, 5])
```

```
In [9]: 1 a1[3:-1]
        Out[9]: array([4, 5])
```

```
In [10]: 1 a1[:2]
        Out[10]: array([1, 2])
```

```
In [11]: 1 a1[3:]
        Out[11]: array([4, 5, 6])
```

1D array - PyTorch

```
In [12]: 1 a1_torch = torch.FloatTensor([1,2,3,4,5,6])  
        2 print(a1_torch)  
tensor([1., 2., 3., 4., 5., 6.])
```

```
In [13]: 1 a1_torch.dim()
```

```
Out[13]: 1
```

```
In [14]: 1 a1_torch.shape
```

```
Out[14]: torch.Size([6])
```

```
In [15]: 1 a1_torch.size()
```

```
Out[15]: torch.Size([6])
```

```
In [16]: 1 a1_torch[0]
```

```
Out[16]: tensor(1.)
```

```
In [17]: 1 a1_torch[1]
```

```
Out[17]: tensor(2.)
```

```
In [18]: 1 a1_torch[-1]
```

```
Out[18]: tensor(6.)
```

```
In [19]: 1 a1_torch[2:5]
```

```
Out[19]: tensor([3., 4., 5.])
```

```
In [20]: 1 a1_torch[3:-1]
```

```
Out[20]: tensor([4., 5.])
```

```
In [21]: 1 a1_torch[:2]
```

```
Out[21]: tensor([1., 2.])
```

```
In [22]: 1 a1_torch[3:]
```

```
Out[22]: tensor([4., 5., 6.])
```

2D array - Numpy

In [23]: ▶

```
1 a2 = np.array([[1,2,3],  
2               [4,5,6],  
3               [7,8,9],  
4               [10,11,12]])
```

In [24]: ▶

```
1 a2
```

```
Out[24]: array([[ 1,  2,  3],  
               [ 4,  5,  6],  
               [ 7,  8,  9],  
               [10, 11, 12]])
```

In [25]: ▶

```
1 a2.ndim
```

```
Out[25]: 2
```

In [26]: ▶

```
1 a2.shape
```

```
Out[26]: (4, 3)
```

2D array - PyTorch

```
In [27]: ▶ 1 a2_torch = torch.FloatTensor([[1,2,3],  
2                                     [4,5,6],  
3                                     [7,8,9],  
4                                     [10,11,12]])
```

```
In [28]: ▶ 1 a2_torch  
Out[28]: tensor([[ 1.,  2.,  3.],  
                [ 4.,  5.,  6.],  
                [ 7.,  8.,  9.],  
                [10., 11., 12.]])
```

```
In [29]: ▶ 1 a2_torch.dim()  
Out[29]: 2
```

```
In [30]: ▶ 1 a2_torch.size()  
Out[30]: torch.Size([4, 3])
```

```
In [31]: ▶ 1 a2_torch[:, 1]  
Out[31]: tensor([ 2.,  5.,  8., 11.])
```

```
In [32]: ▶ 1 a2_torch[:, 1].size()  
Out[32]: torch.Size([4])
```

```
In [33]: ▶ 1 a2_torch[:, :-1]  
Out[33]: tensor([[ 1.,  2.],  
                [ 4.,  5.],  
                [ 7.,  8.],  
                [10., 11.]])
```

차원의 차이

```
In [34]: 1 a3 = torch.FloatTensor([1,2])  
          2 print(a3)  
  
          tensor([1., 2.])
```

```
In [35]: 1 a3.dim()
```

```
Out[35]: 1
```

```
In [36]: 1 a3.size()
```

```
Out[36]: torch.Size([2])
```

```
In [37]: 1 a4 = torch.FloatTensor([[1,2]])  
          2 print(a4)  
  
          tensor([[1., 2.]])
```

```
In [38]: 1 a4.dim()
```

```
Out[38]: 2
```

```
In [39]: 1 a4.size()
```

```
Out[39]: torch.Size([1, 2])
```

AI/BigData 인텐시브 과정

Section 1. PyTorch 기초

Section 1-2. PyTorch 기초 연산

벡터 + 벡터

```
In [40]: 1 v1 = torch.FloatTensor([[3,3]])
```

```
In [41]: 1 v1
```

```
Out[41]: tensor([[3., 3.]])
```

```
In [42]: 1 v2 = torch.FloatTensor([5,5])
```

```
In [43]: 1 v2
```

```
Out[43]: tensor([5., 5.])
```

```
In [44]: 1 v1 + v2
```

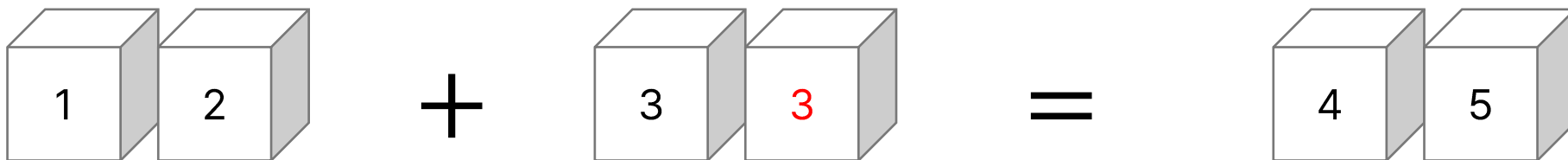
```
Out[44]: tensor([[8., 8.]])
```

벡터 + 스칼라

```
In [45]: ▶ 1 v3 = torch.FloatTensor([[1,2]])  
          2 v4 = torch.FloatTensor([3])
```

```
In [46]: ▶ 1 v3 + v4
```

```
Out[46]: tensor([[4., 5.]])
```

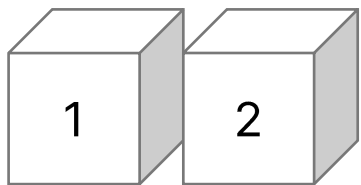


차원이 다른 벡터의 합

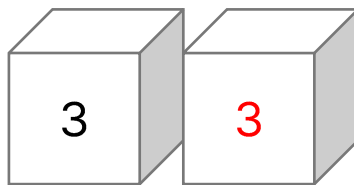
```
In [47]: ▶ 1 v5 = torch.FloatTensor([[1,2]])  
          2 v6 = torch.FloatTensor([[3], [4]])
```

```
In [48]: ▶ 1 v5 + v6
```

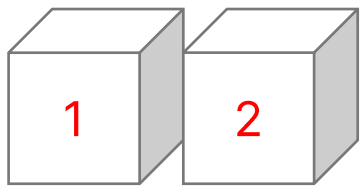
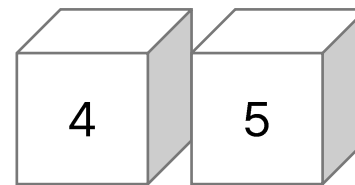
```
Out[48]: tensor([[4., 5.],  
                 [5., 6.]])
```



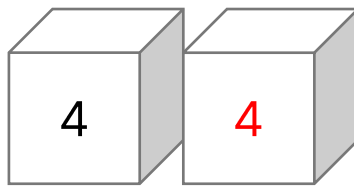
+



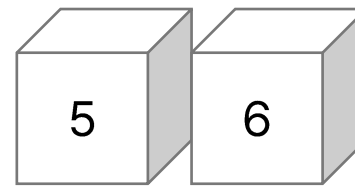
=



+



=



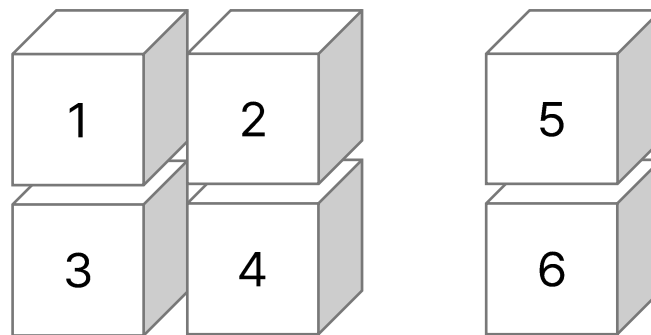
행렬 곱

```
In [49]: ▶ 1 m1 = torch.FloatTensor([[1,2],  
2          [3,4]])
```

```
In [50]: ▶ 1 m2 = torch.FloatTensor([[5],  
2          [6]])
```

```
In [51]: ▶ 1 m1.matmul(m2)
```

```
Out[51]: tensor([[17.],  
                [39.]])
```



행렬 원소곱

```
In [52]: 1 m3 = torch.FloatTensor([[1,2],  
2                                     [3,4]])
```

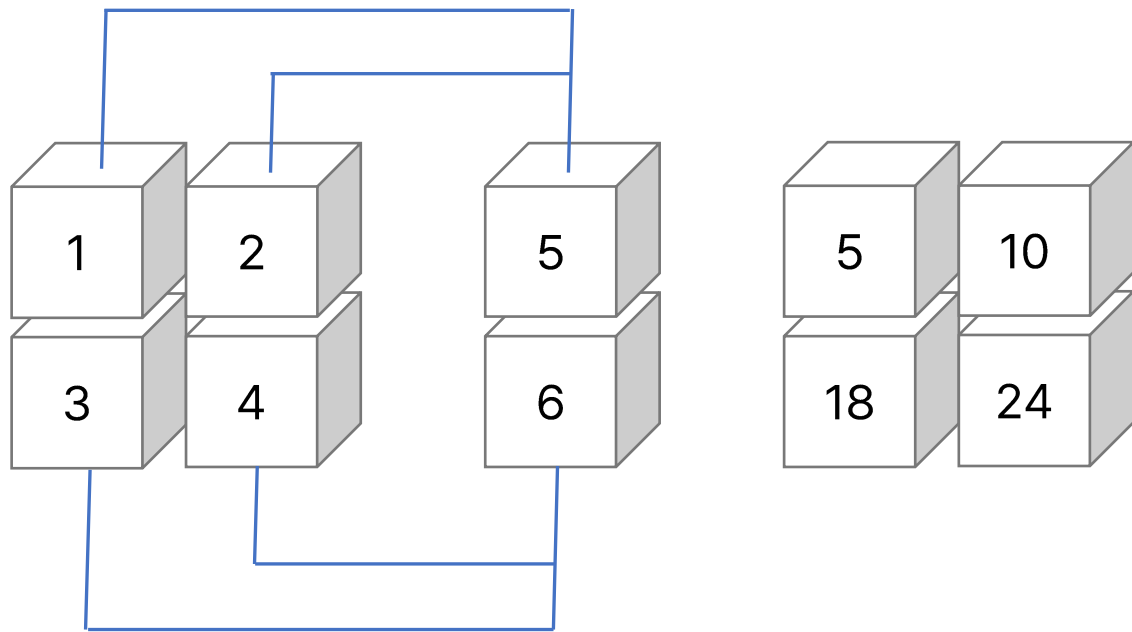
```
In [53]: 1 m4 = torch.FloatTensor([[5],  
2                                   [6]])
```

```
In [54]: 1 m3*m4
```

```
Out[54]: tensor([[ 5., 10.],  
                [18., 24.]])
```

```
In [55]: 1 m3.mul(m4)
```

```
Out[55]: tensor([[ 5., 10.],  
                [18., 24.]])
```



Mean

```
In [56]: 1 m1 = torch.FloatTensor([[1,2],  
2          [3,4]])
```

```
In [57]: 1 m1.mean()
```

```
Out[57]: tensor(2.5000)
```

$$\frac{1 + 2 + 3 + 4}{4} = 2.5$$

```
In [58]: 1 m1.mean(dim=0)
```

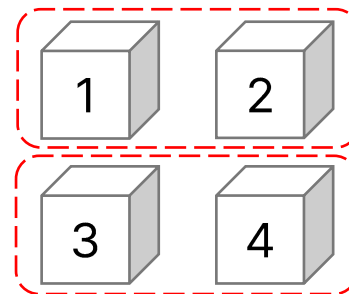
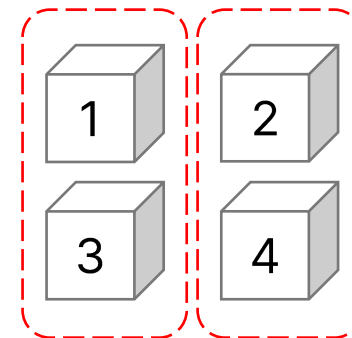
```
Out[58]: tensor([2., 3.])
```

```
In [59]: 1 m1.mean(dim=1)
```

```
Out[59]: tensor([1.5000, 3.5000])
```

```
In [60]: 1 m1.mean(dim=-1)
```

```
Out[60]: tensor([1.5000, 3.5000])
```



Sum

```
In [61]: 1 m1 = torch.FloatTensor([[1,2],  
2          [3,4]])
```

```
In [62]: 1 m1.sum()
```

```
Out[62]: tensor(10.)
```

```
In [63]: 1 m1.sum(dim=0)
```

```
Out[63]: tensor([4., 6.])
```

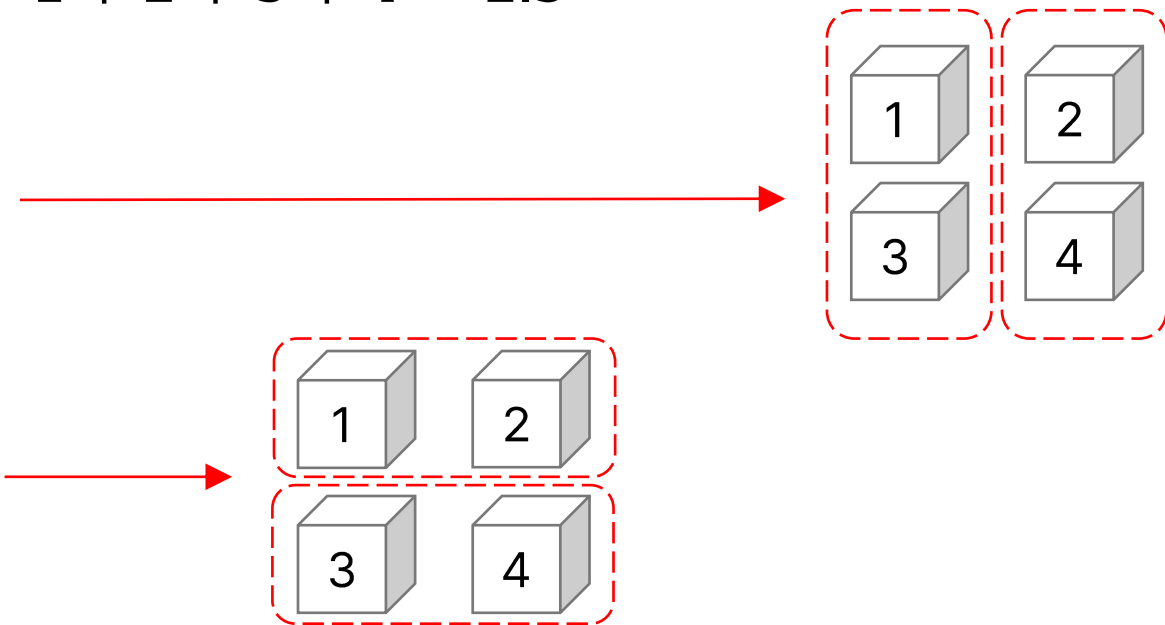
```
In [64]: 1 m1.sum(dim=1)
```

```
Out[64]: tensor([3., 7.])
```

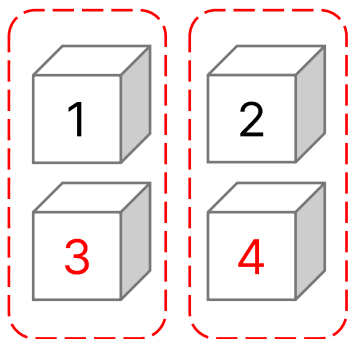
```
In [65]: 1 m1.sum(dim=-1)
```

```
Out[65]: tensor([3., 7.])
```

$$1 + 2 + 3 + 4 = 2.5$$

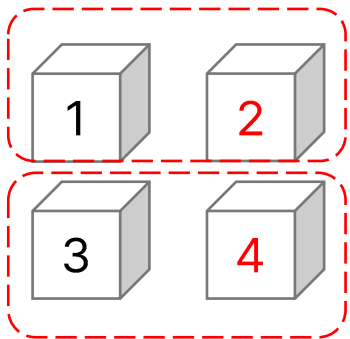


Max & Argmax



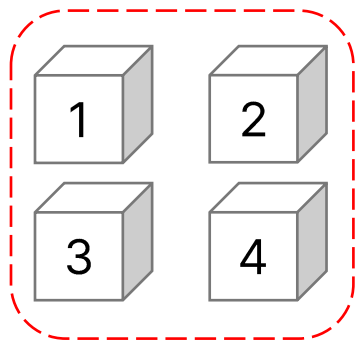
$$\max(1, 3) = 3$$

$$\max(2, 4) = 4$$



$$\max(1, 2) = 2$$

$$\max(3, 4) = 4$$



```
In [66]: 1 m1 = torch.FloatTensor([[1,2],
2                                     [3,4]])
```

```
In [67]: 1 m1.max()
```

```
Out[67]: tensor(4.)
```

```
In [68]: 1 m1.max(dim=0)
```

```
Out[68]: torch.return_types.max(
  values=tensor([3., 4.]),
  indices=tensor([1, 1]))
```

```
In [69]: 1 print("Max =", m1.max(dim=0)[0])
```

```
Max = tensor([3., 4.])
```

```
In [70]: 1 print("Argmax =", m1.max(dim=0)[1])
```

```
Argmax = tensor([1, 1])
```

```
In [71]: 1 m1.max(dim=1)
```

```
Out[71]: torch.return_types.max(
  values=tensor([2., 4.]),
  indices=tensor([1, 1]))
```

```
In [72]: 1 print("Max =", m1.max(dim=1)[0])
```

```
Max = tensor([2., 4.])
```

```
In [73]: 1 print("Argmax =", m1.max(dim=1)[1])
```

```
Argmax = tensor([1, 1])
```

AI/BigData 인텐시브 과정

Section 1. PyTorch 기초

Section 1-3. 행렬 변환

View

```
In [3]: 1 t = [[1, 2, 3, 4],
2          [5, 6, 7, 8],
3          [9, 10, 11, 12]],
4
5          [[13, 14, 15, 16],
6          [17, 18, 19, 20],
7          [21, 22, 23, 24]]
```

```
In [4]: 1 ft = torch.FloatTensor(t)
2 print(ft.shape)
torch.Size([2, 3, 4])
```

```
In [6]: 1 ft.size()
```

```
Out[6]: torch.Size([2, 3, 4])
```

```
In [5]: 1 ft.view([-1, 3])
```

```
Out[5]: tensor([[ 1.,  2.,  3.],
                [ 4.,  5.,  6.],
                [ 7.,  8.,  9.],
                [10., 11., 12.],
                [13., 14., 15.],
                [16., 17., 18.],
                [19., 20., 21.],
                [22., 23., 24.]])
```

```
In [7]: 1 ft.view([-1, 3]).shape
```

```
Out[7]: torch.Size([8, 3])
```

```
In [8]: 1 ft.view([-1, 1, 3])
```

```
Out[8]: tensor([[[ 1.,  2.,  3.],
                  [ 4.,  5.,  6.],
                  [ 7.,  8.,  9.],
                  [10., 11., 12.],
                  [13., 14., 15.],
                  [16., 17., 18.],
                  [19., 20., 21.],
                  [22., 23., 24.]])])
```

```
In [9]: 1 ft.view([-1, 1, 3]).shape
```

```
Out[9]: torch.Size([8, 1, 3])
```

squeeze

차원이 1인 경우 없애줌

```
In [8]: 1 ft = torch.FloatTensor([[0], [1], [2]])
```

```
In [9]: 1 ft
```

```
Out[9]: tensor([[0.],  
               [1.],  
               [2.]])
```

```
In [10]: 1 ft.shape
```

```
Out[10]: torch.Size([3, 1])
```

```
In [11]: 1 ft.squeeze()
```

```
Out[11]: tensor([0., 1., 2.])
```

```
In [12]: 1 ft.squeeze().shape
```

```
Out[12]: torch.Size([3])
```

unsqueeze

차원 1을 추가해줌

```
In [13]: 1 ft = torch.Tensor([1, 2, 3])
          2 print(ft)
          3 print(ft.shape)

tensor([1., 2., 3.])
torch.Size([3])
```

```
In [14]: 1 ft.unsqueeze(0) 0번째 차원에 1을 넣어라

Out[14]: tensor([[1., 2., 3.]])
```

```
In [15]: 1 ft.unsqueeze(0).shape

Out[15]: torch.Size([1, 3]) 0번째 차원에 1이 추가됨
```

```
In [16]: 1 ft.view(1, -1)

Out[16]: tensor([[1., 2., 3.]])
```

```
In [17]: 1 ft.view(1, -1).shape

Out[17]: torch.Size([1, 3])
```

```
In [18]: 1 ft.unsqueeze(1) 1번째 차원에 1을 넣어라

Out[18]: tensor([[1.],
                  [2.],
                  [3.]])
```

```
In [19]: 1 ft.unsqueeze(1).shape

Out[19]: torch.Size([3, 1]) 1번째 차원에 1이 추가됨
```

```
In [20]: 1 ft.unsqueeze(-1)

Out[20]: tensor([[1.],
                  [2.],
                  [3.]])
```

```
In [21]: 1 ft.unsqueeze(-1).shape

Out[21]: torch.Size([3, 1])
```

Type Casting

```
In [22]: ▶ 1 lt = torch.LongTensor([1, 2, 3, 4])  
          2 print(lt)  
  
          tensor([1, 2, 3, 4])
```

```
In [23]: ▶ 1 lt.float()  
  
Out[23]: tensor([1., 2., 3., 4.])
```

```
In [24]: ▶ 1 bt = torch.ByteTensor([True, False, False, True])  
          2 print(bt)  
  
          tensor([1, 0, 0, 1], dtype=torch.uint8)
```

```
In [25]: ▶ 1 bt.long()  
  
Out[25]: tensor([1, 0, 0, 1])
```

```
In [26]: ▶ 1 bt.float()  
  
Out[26]: tensor([1., 0., 0., 1.])
```

concatenate

```
In [27]: ▶ 1 x = torch.FloatTensor([[1, 2], [3, 4]])  
          2 y = torch.FloatTensor([[5, 6], [7, 8]])
```

```
In [28]: ▶ 1 torch.cat([x, y], dim=0)
```

```
Out[28]: tensor([[1., 2.],  
                 [3., 4.],  
                 [5., 6.],  
                 [7., 8.]])
```

```
In [29]: ▶ 1 torch.cat([x, y], dim=1)
```

```
Out[29]: tensor([[1., 2., 5., 6.],  
                 [3., 4., 7., 8.]])
```

stacking

```
In [30]: 1 x = torch.FloatTensor([1, 4])  
        2 y = torch.FloatTensor([2, 5])  
        3 z = torch.FloatTensor([3, 6])
```

```
In [31]: 1 torch.stack([x, y, z])
```

```
Out[31]: tensor([[1., 4.],  
                [2., 5.],  
                [3., 6.]])
```

```
In [32]: 1 torch.stack([x, y, z], dim=1)
```

```
Out[32]: tensor([[1., 2., 3.],  
                [4., 5., 6.]])
```

```
In [33]: 1 torch.cat([x.unsqueeze(0), y.unsqueeze(0), z.unsqueeze(0)], dim=0)
```

```
Out[33]: tensor([[1., 4.],  
                [2., 5.],  
                [3., 6.]])
```

ones & zeros

```
In [34]: ▶ 1 x = torch.FloatTensor([[0, 1, 2], [2, 1, 0]])  
          2 print(x)
```

```
tensor([[0., 1., 2.],  
        [2., 1., 0.]])
```

```
In [35]: ▶ 1 torch.ones_like(x)
```

```
Out[35]: tensor([[1., 1., 1.],  
                 [1., 1., 1.]])
```

```
In [36]: ▶ 1 torch.zeros_like(x)
```

```
Out[36]: tensor([[0., 0., 0.],  
                 [0., 0., 0.]])
```

In place operation

```
In [37]: 1 x = torch.FloatTensor([[1, 2], [3, 4]])
```

```
In [38]: 1 x.mul(2)
```

```
Out[38]: tensor([[2., 4.],  
                [6., 8.]])
```

```
In [39]: 1 x
```

```
Out[39]: tensor([[1., 2.],  
                [3., 4.]])
```

```
In [40]: 1 x.mul_(2)
```

```
Out[40]: tensor([[2., 4.],  
                [6., 8.]])
```

```
In [41]: 1 x
```

```
Out[41]: tensor([[2., 4.],  
                [6., 8.]])
```

감사합니다.

Q & A